An Explainable, Self-Healing Deep Learning Framework for Enterprise Network Fault Diagnosis, Prediction and Recovery

Jifano Oliver J P
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
Reg. No. 24BCE2742

Hrithikesh Vineeth Nair
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
Reg. No. 24BAI0266

T. Nitin Yajatkumar
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India
Reg. No. 24BCE0571

***Abstract*—Enterprise networks suffer frequent faults such as link errors, congestion, provider outages, DNS failures and wireless interference, and are still largely repaired by hand. This paper presents NetSage, a framework that diagnoses, predicts, explains and automatically recovers from such faults. A deep neural network classifies twelve telemetry features into eight network states with 99.56% test accuracy, and a second network estimates the risk of a fault within the next fifteen seconds with 99.25% precision. Every diagnosis is explained with integrated gradients and plain-language evidence. Before any recovery action is applied, it is executed on a continuously synchronised digital twin of the network and checked against seven safety constraints; only actions predicted to be both effective and safe are executed, then verified and rolled back if they fail. Evaluated on a simulated campus network with seven fault types and on a live Wi-Fi connection, NetSage repaired every injected fault, prevented slowly developing faults before users were affected, rejected at least one unsafe or ineffective action in every scenario, and raised no false alarms.**

Keywords—network fault diagnosis, deep learning, explainable AI, digital twin, self-healing networks, fault prediction

I. Introduction

Organisations depend on their networks for every business process, and a single failed link, overloaded switch or crashed DNS server can stop hundreds of users. Conventional network management raises alarms when a metric crosses a fixed threshold; an engineer must then interpret the alarms, locate the root cause and choose a fix. This process is slow, depends on individual expertise and is risky: a plausible action such as failing over to a backup link can overload that link and cause a second outage.

Machine learning can recognise fault patterns that fixed rules miss, but two obstacles prevent it from repairing networks automatically. Deep models are opaque, so operators cannot verify their decisions, and letting a model act directly on a production network is unsafe without a way to check the consequences first. NetSage addresses both obstacles. Its contributions are:

- a compact twelve-feature telemetry representation shared by a simulated enterprise network and a live Wi-Fi connection;
- two deep neural networks for fault diagnosis (eight classes) and short-term fault prediction;
- per-decision explanations using integrated gradients, evidence sentences and per-action reasons;
- a synchronised digital twin that evaluates every candidate recovery action against seven safety constraints before execution;
- a closed self-healing loop with confirmation, verification, rollback, escalation and optional operator approval.

II. Related Work

Classical fault management collects device counters, for example through SNMP, and alarms on static thresholds. It is transparent but cannot separate faults with overlapping symptoms and gives no early warning. Machine-learning approaches learn the mapping from telemetry to fault type; deep networks trained with Adam [1] and He initialisation [2] are a standard choice, but most systems stop at diagnosis. Integrated gradients [3] attribute a prediction to its inputs relative to a baseline and satisfy completeness, which makes them suitable for explaining individual diagnoses. A digital twin is a virtual representation kept synchronised with a physical system and used to analyse its behaviour [4]; network digital twins have been proposed for validating changes and what-if scenarios before deployment [5]. NetSage combines these ideas in one loop and uses the twin specifically as a safety gate for automatic recovery.

III. System Architecture

NetSage runs one engine loop per sample (Fig. 1). A data source, either the simulated campus or the live Wi-Fi monitor, produces the same twelve-feature vector, so the same trained models serve both modes. The diagnoser and predictor turn it into a diagnosis and a risk estimate, the explainability module attributes them to the inputs, the digital twin evaluates recovery options, and the self-healing engine executes and verifies the chosen action. A Flask server exposes the state through a REST API and a dashboard that refreshes every second.

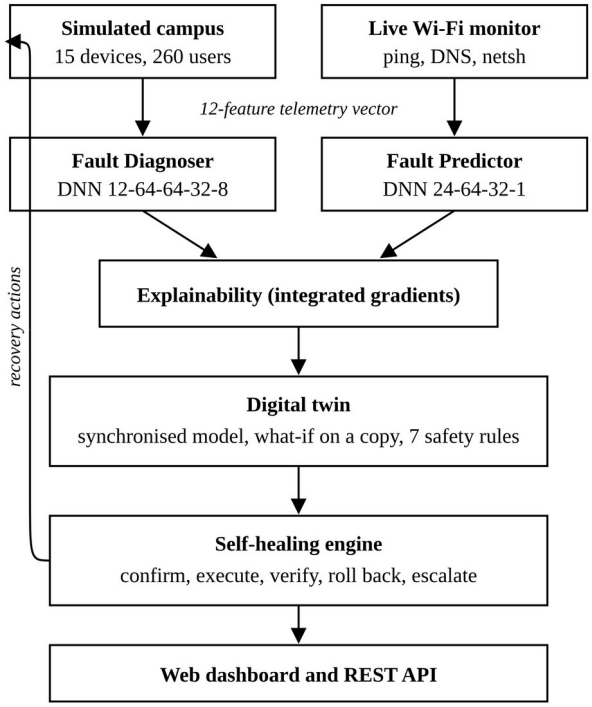


Fig. 1. NetSage architecture.

IV. Methodology

A. Telemetry and Fault Classes

Each sample contains the features in Table I. Unanswered probes are recorded as a 1000 ms timeout and 100% loss. Latencies are log-scaled, the latency ratio is log-scaled up to 20x and percentages are divided by 100, giving a fixed normalisation independent of the training data. The eight classes are healthy, link down, packet loss, congestion, upstream outage, DNS failure, wireless degraded and device overload.

TABLE I. Telemetry Features

Feature	Meaning
Gateway latency / loss	Round-trip time and loss to the default gateway
Internet latency / loss	Round-trip time and loss to an internet host
Jitter	Variation of internet latency
Latency ratio	Internet latency / healthy baseline
DNS failure rate	Share of real DNS queries failing
Wi-Fi signal	Received signal quality (%)
Link state	Access link associated (0/1)
Utilisation	Load on the busiest link of the path
Interface errors	Physical-layer error rate
Device CPU	CPU of the busiest device on the path

B. Dataset

Labelled fault data from real enterprise networks is not publicly available, so a synthetic dataset was generated from documented fault signatures. Each sample starts from a randomised healthy link with an internet baseline between 3 ms (campus LAN) and 120 ms (mobile hotspot) and applies a class signature, for example 2.2-15x latency with high utilisation for congestion, or a responsive gateway with 90-100% internet loss for an upstream outage. Class ranges overlap and measurement noise is added so the task is not trivially separable. The diagnoser uses 3,000 samples per class (24,000 in total, split 80/20). The predictor uses 2,500 time series containing healthy periods, gradual build-ups of 10-30 samples, abrupt failures and short benign spikes, cut into 100,000 sliding windows. Both datasets are exported as CSV files for inspection.

C. Deep Neural Networks

Both models are fully connected networks implemented from first principles in NumPy with He initialisation, ReLU hidden units, cross-entropy loss and mini-batch Adam with cosine learning-rate decay and L2 regularisation. The **diagnoser** (12-64-64-32-8, 7,336 parameters) takes the median of the last three samples and outputs class probabilities through a softmax. The **predictor** (24-64-32-1, 3,713 parameters) takes the recent level and the trend of each feature over ten samples and outputs, through a sigmoid, the probability that a fault is active within the next fifteen samples. Class-balanced weights compensate for the smaller share of fault windows.

D. Explainability

For a diagnosis with predicted class c, NetSage computes integrated gradients [3] from a healthy reference link x' to the measurement x with $m = 32$ steps:

$$IG_i(x) = (x_i - x'_i)\cdot(1/m)\,\Sigma_k\,\partial F_c(x' + (k/m)(x - x')) / \partial x_i \quad (1)$$

where F is the softmax output for class c. Input gradients are obtained by back-propagating through the network. The dashboard ranks features by their share of the total attribution and adds class-specific evidence sentences that quote the measurements, for example that the gateway answers while all internet probes are lost, which places the fault beyond the gateway.

E. Fault Prediction

An early warning is raised when the predictor's risk stays at or above 0.7 for three consecutive samples while the network is still healthy. To identify the approaching fault, every feature is extrapolated fifteen samples ahead with the Theil-Sen estimator [6], [7], the median of all pairwise slopes, which is robust to single noisy samples, and the extrapolated state is classified by the diagnoser.

F. Digital Twin

The twin is an analytical model of the campus: three user groups (260 users) on three access points, two access switches, two redundant core switches, a data-centre switch with two DNS servers and an ERP application, an edge router and two providers (1 Gb/s primary, 500 Mb/s backup). Routing uses Dijkstra's algorithm [8] on configured link costs, including static default routes, provider load-sharing and QoS policing. For each link with load L and capacity C, utilisation and queueing delay are

$$u = L / C, \quad d_q = 1.5\,u / (1 - u)\ \text{ms} \quad (2)$$

capped at 60 ms, with overflow loss above full load. The model also computes device CPU from forwarded traffic and control-plane storms, Wi-Fi signal, airtime load and co-channel interference, internal and external DNS resolution per group, and redundancy as the number of edge-disjoint paths from each access switch to the edge. The twin is re-synchronised every second and reports its fidelity, the agreement between predicted and measured latency, which stayed at 95-96%. For early warnings it extrapolates fault drivers (traffic, CPU storm, interference, error rate) to evaluate preventive actions on the predicted future network.

G. Safety-Constrained Self-Healing

For each fault class a playbook proposes three candidate actions, deliberately including tempting but dangerous ones. Each candidate is applied to a copy of the twin and rated RECOMMENDED, SAFE, UNSAFE (fixes the symptom but violates a constraint), INEFFECTIVE (does not fix it) or BLOCKED (precondition fails). The constraints are: every group keeps internet, ERP and DNS reachability; loss of at most 2% and latency of at most 100 ms; no link above 85% utilisation; no device CPU above 85%; no access point over capacity or on a clashing channel; no access switch loses redundancy; and disruption within 60 user-minutes. Safe actions are ranked by disruption, worst link utilisation and known risks.

The control loop is shown in Fig. 2. A fault is confirmed after three identical diagnoses. If the diagnosed playbook has no safe action, the remaining playbooks are searched in order of diagnosis probability. After execution the engine requires three consecutive healthy samples within a verification window; otherwise it rolls back and tries the next safe action, and escalates to the operator when none remains. With automatic healing disabled, it stops at the recommendation and waits for approval.

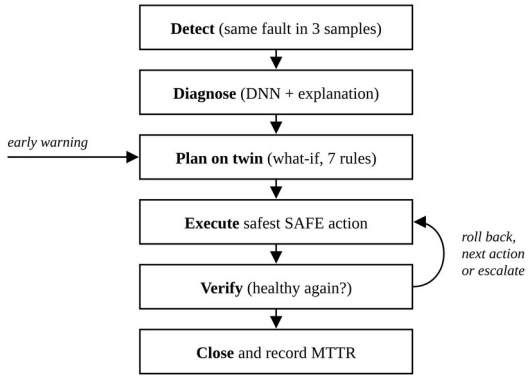


Fig. 2. Self-healing control loop.

H. Fault Injection

The simulated campus provides seven faults: four develop gradually over 12-25 samples (backup traffic saturating the primary provider, a broadcast storm on a core switch, fibre errors on an uplink and radio interference on an access point) and three are abrupt (provider outage, DNS service crash and access-point failure). In live mode, faults are created on the host computer only with standard Windows mechanisms: outbound firewall rules for packet loss, internet outage and DNS failure, netsh for Wi-Fi disconnection and adapter shutdown, and parallel downloads for congestion. Each live fault is undone automatically. The live twin models host policy, adapter, access point, gateway, upstream and DNS, and may only remove rules created by NetSage, stay on the same authorised network and interrupt the connection for at most ten seconds.

V. Implementation

NetSage is written in Python (about 3,500 lines in 21 modules) with NumPy for all numerical work and Flask for the web server. The dashboard uses plain HTML, CSS and JavaScript with canvas charts and an SVG topology, so it works offline. Windows integration uses netsh, ping, ipconfig and the Native Wi-Fi API (wlanapi.dll) through ctypes, including pinning the adapter to a chosen access point. An automated suite of 16 tests covers gradient correctness, model accuracy, twin decisions, the closed loop and the live loop against a mocked Windows network; all tests pass.

VI. Results and Discussion

A. Model Performance

TABLE II. Model Performance on Held-Out Data

Model / metric	Value
Diagnoser test accuracy (4,800 samples)	99.56%
Lowest per-class recall (packet loss)	98.4%
Predictor accuracy (20,000 windows)	94.70%
Predictor precision	99.25%
Predictor recall	86.97%
Training time, both networks (laptop CPU)	about 60 s

Remaining diagnosis errors occur between classes with genuinely overlapping symptoms, such as packet loss versus wireless degradation. The predictor is tuned for precision so that preventive actions are rarely triggered without cause; its recall reflects that abrupt failures cannot be predicted. Because the data is synthetic and controlled, accuracy on real enterprise telemetry is expected to be lower.

B. Twin-Verified Recovery

TABLE III. Recovery Decisions of the Digital Twin

Fault	Chosen action	Rejected (reason)
Backup saturates ISP-A	QoS policing of bulk traffic	All traffic to ISP-B (overload, 38% loss)
Storm on core switch	Control-plane policing	Shut switch (redundancy lost)
Fibre errors on uplink	Drain to redundant link	Shut link (single point of failure); bounce (errors persist)
AP interference	Move to clean channel	Channel clashing with AP-2; max TX power
ISP-A outage	Fail over to ISP-B with QoS	Plain failover (ISP-B at 99%); reboot edge (ineffective)
DNS-1 crash	Fail over to DNS-2	Public DNS (internal names fail); restart (195 user-min)
AP-1 failure	Split clients over AP-2 and AP-3	All to AP-2 (over capacity); reboot (unreachable)

Each fault was injected after 15 healthy samples and observed for 60 samples, with predictive healing on and off (14 runs). All runs ended with a healthy network. With prediction enabled, the backup-traffic congestion and the broadcast storm were prevented before the diagnosis turned faulty; faster build-ups and abrupt faults were repaired reactively. Faults were diagnosed about 1-2.5 s after injection and service was restored within a few seconds of detection, for example 3 s for a provider outage. In every scenario the twin rejected at least one plausible action that would have overloaded a link, broken internal name resolution, removed redundancy, overloaded an access point or exceeded the disruption budget. A 150-sample run without faults produced no incidents. In live mode, packet loss, internet outage, DNS failure, disconnection and adapter shutdown were each diagnosed correctly and repaired. Fig. 3 shows the dashboard after a predicted congestion was prevented, and Fig. 4 the twin's evaluation of an interference incident.

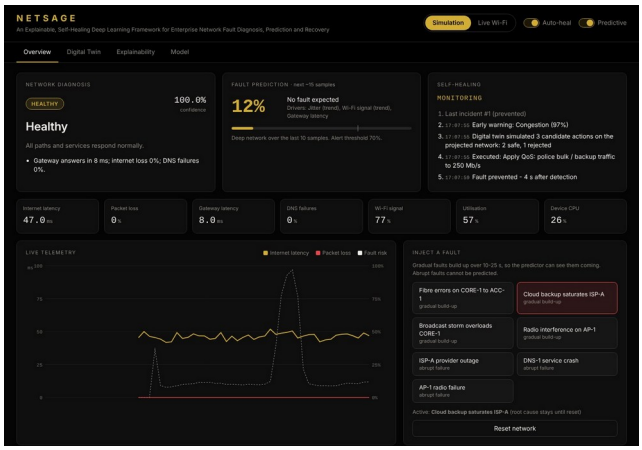


Fig. 3. Dashboard after a predicted congestion was prevented.

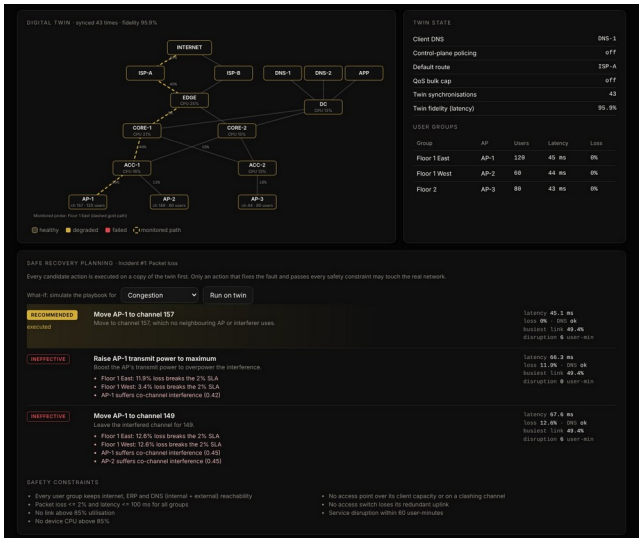


Fig. 4. Digital twin topology and evaluated recovery actions.

C. Limitations

The models are trained on synthetic data and the campus is an analytical model rather than a physical network; both were chosen to demonstrate the full loop safely and repeatably. Live mode runs on Windows and acts only on the host computer, and abrupt failures can be detected but not predicted.

VII. Conclusion and Future Work

NetSage shows that automatic network recovery can be both intelligent and trustworthy. Deep networks diagnose faults with 99.56% accuracy and anticipate gradual faults; integrated gradients make each decision transparent; and a synchronised digital twin acts as a safety gate that rejected unsafe or ineffective actions in every tested scenario while selecting one that restored service. Future work includes training on real enterprise telemetry, sequence models such as LSTMs or Transformers for longer-range prediction, connecting the twin and actions to real devices through SNMP, NETCONF or SDN controllers, learning twin parameters from measurements, and constrained reinforcement learning for discovering new recovery policies.

Acknowledgment

The authors thank Arivoli A, School of Computer Science and Engineering, Vellore Institute of Technology, for guidance throughout this project.

References

[1] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proc. Int. Conf. Learning Representations (ICLR), 2015.

[2] K. He, X. Zhang, S. Ren, and J. Sun, “Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification,” in Proc. IEEE Int. Conf. Computer Vision (ICCV), 2015, pp. 1026–1034.

[3] M. Sundararajan, A. Taly, and Q. Yan, “Axiomatic attribution for deep networks,” in Proc. Int. Conf. Machine Learning (ICML), 2017, pp. 3319–3328.

[4] M. Grieves and J. Vickers, “Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems,” in Transdisciplinary Perspectives on Complex Systems. Cham, Switzerland: Springer, 2017, pp. 85–113.

[5] C. Zhou et al., “Network digital twin: Concepts and reference architecture,” IRTF Network Management Research Group, Internet-Draft draft-irtf-nmrg-network-digital-twin-arch.

[6] H. Theil, “A rank-invariant method of linear and polynomial regression analysis,” Proc. Royal Netherlands Academy of Sciences, vol. 53, 1950.

[7] P. K. Sen, “Estimates of the regression coefficient based on Kendall's tau,” J. American Statistical Association, vol. 63, no. 324, pp. 1379–1389, 1968.

[8] E. W. Dijkstra, “A note on two problems in connexion with graphs,” Numerische Mathematik, vol. 1, pp. 269–271, 1959.